

Documentazione del Progetto BlackJack OG

- **Autori**
 - Onofrio Soranno
 - Luca Perrone
- **Sommario**
 1. Analisi
 2. Progettazione
 3. Codifica
 4. Test
 5. Conclusioni

Analisi

Descrizione Generale:

Il progetto implementa un gioco di BlackJack con interfaccia a riga di comando, gestione degli utenti, salvataggio delle partite e storico delle giocate. Il sistema permette agli utenti di registrarsi, effettuare il login, giocare a BlackJack con puntate variabili e visualizzare le ultime partite giocate.

Requisiti Funzionali

Codice	Nome	Descrizione
R01	Registrazione	L'utente può registrarsi con nome, cognome, username e password.
R02	Login	L'utente può accedere al sistema con username e password.
R03	Gestione Saldo	Il sistema gestisce il saldo dell'utente durante le partite.
R04	Nuova Partita	L'utente può iniziare una nuova partita di BlackJack.
R05	Carica Partita	L'utente può riprendere una partita precedentemente salvata.
R06	Visualizzazione Mani	L'utente può visualizzare le ultime mani giocate.

R07	Puntate	L'utente può effettuare puntate di valore predefinito (10, 20, 50, 100, 200).
R08	Gestione Mazzo	Il sistema gestisce il mazzo di carte, incluso il mescolamento.
R09	Salvataggio Partita	Il sistema salva automaticamente lo stato della partita.

Casi d'Uso

- **Registrazione**
 - **Pre-condizioni:** Nessuna
 - **Post-condizioni:**
 - **Successo:** L'utente è registrato e loggato automaticamente
 - **Fallimento:** Messaggio di errore (username già usato/password non valida)
 - **Scenario di base:** L'utente inserisce dati validi e conferma la password
 - **Scenario alternativo:** Username già esistente o password < 6 caratteri
- **Login**
 - **Pre-condizioni:** nessuna
 - **Post-condizioni:**
 - **Successo:** l'utente accede al sistema
 - **Fallimento:** viene mostrato un messaggio di errore
 - **Scenario di base:** L'utente inserisce username e password corretti
 - **Scenario alternativo:** Credenziali errate
- **Nuova Partita**
 - **Pre-condizioni:** utente loggato
 - **Post-condizioni:**
 - **Successo:** inizia una nuova partita
 - **Fallimento:** messaggio di errore
 - **Scenario di base:** L'utente seleziona "Nuova partita"
 - **Scenario alternativo:** Saldo insufficiente
- **Continua Partita**
 - **Pre-condizioni:** Partita terminata, saldo > 0
 - **Post-condizioni:**
 - **Successo (Si):** Nuova mano con lo stesso mazzo
 - **Successo (No):** Ritorno al menu principale
 - **Scenario di base:** L'utente sceglie "S" per continuare
 - **Scenario alternativo:** Input non valido (richiesta ripetuta)

Progettazione

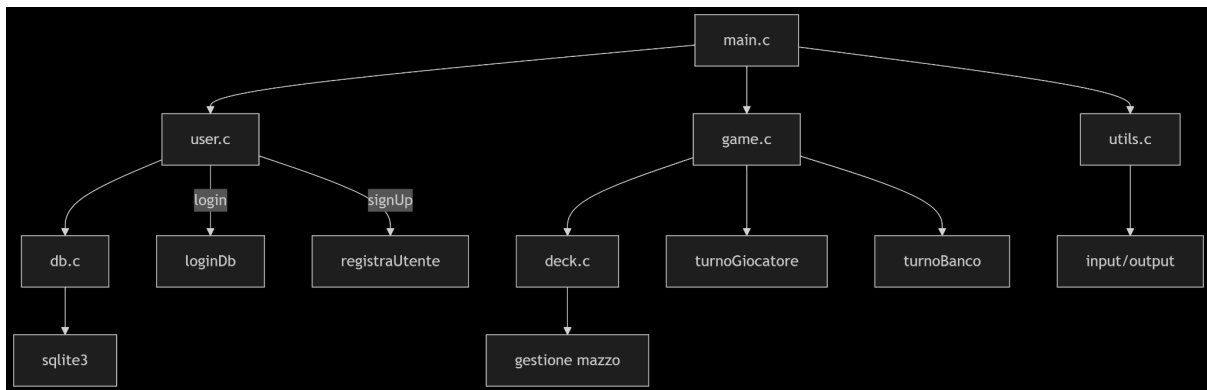
Principali Variabili, Strutture Dati e File.

Nome	Tipologia	Descrizione	Tipo/Campi/Valori
Utente	struct	Contiene i dati dell'utente	id, nome, cognome, username, saldo
Carta	struct	Rappresenta una carta da gioco	valore, seme, punti, next
blackjack.db	File SQLite	Database per persistenza dati	Tabelle: utenti, blackjack_hands

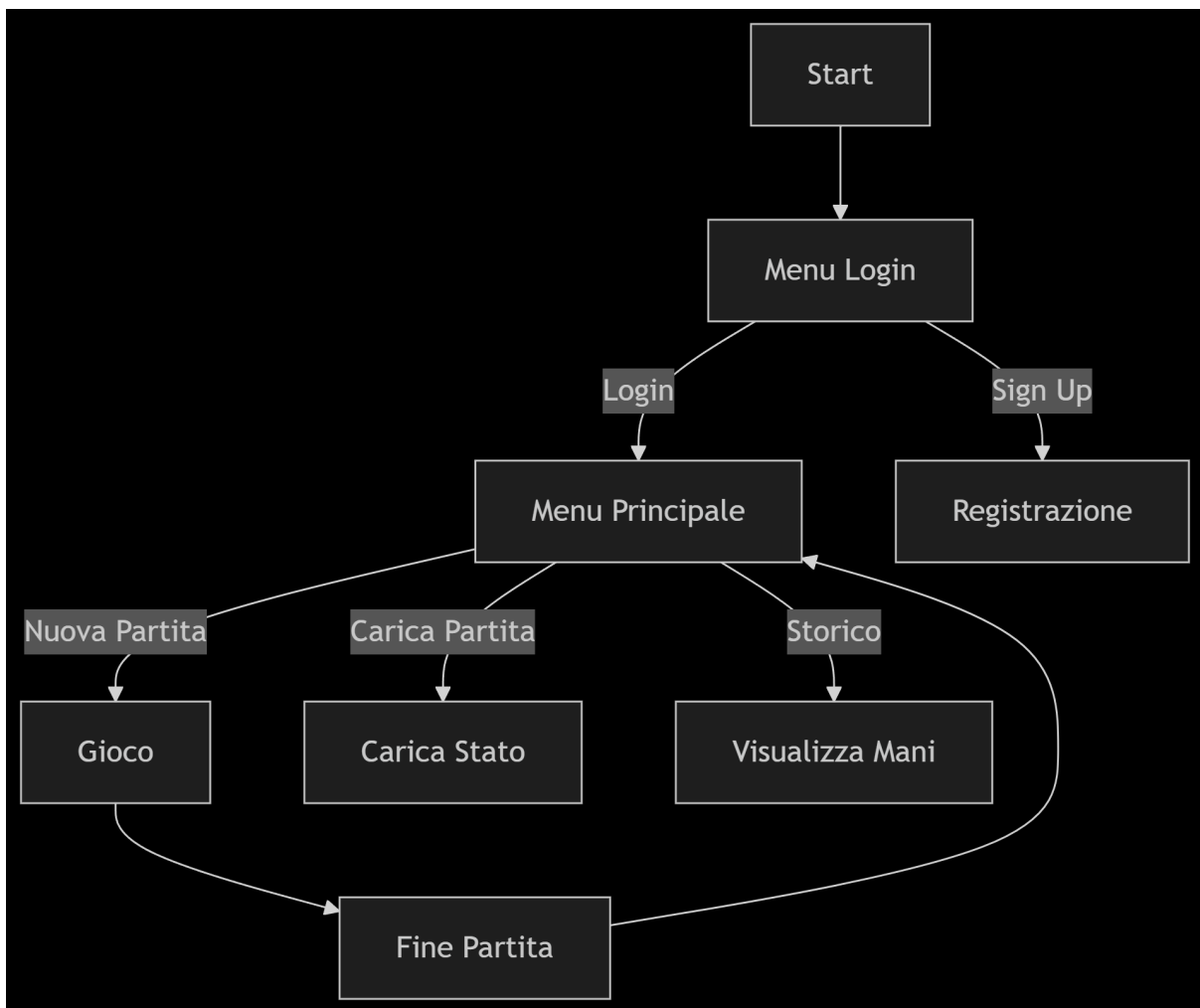
Librerie e Funzioni Principali

- **utils.c**
 - *getPassword()*: Legge una password senza mostrarla a schermo
 - *printCentered()*: Stampa un messaggio centrato nella console
 - *getScelta()*: Legge la scelta dell'utente dal menu
- **user.c**
 - *login()*: Gestisce il processo di login
 - *signUp()*: Gestisce la registrazione di un nuovo utente
- **game.c**
 - *partita()*: Gestisce il flusso principale del gioco
 - *turnoGiocatore()*: Gestisce le mosse del giocatore
 - *turnoBanco()*: Gestisce le mosse automatiche del banco
- **deck.c**
 - *creaMazzo()*: Crea un mazzo di carte
 - *mescolaMazzo()*: Mescola il mazzo di carte
 - *contaCarte()*: Conta le carte rimanenti nel mazzo
- **db.c**
 - *apriDatabase()*: Apre la connessione al database
 - *creaTabellaUser()*: Crea la tabella utenti se non esiste
 - *salvaMazzoUtente()*: Salva lo stato del mazzo per un utente

Dipendenze tra Funzioni



Flowchart principale



Codifica

Frammenti di Codice Significativi:

- Gestione del Mazzo (deck.c)

```
void creaMazzo(Carta** top) {
    const char* semi[] = { "♠", "♥", "♦", "♣" };
    char* valori[] = { "A", "2", "3", "4", "5", "6", "7", "8", "9", "10", "J",
        "Q", "K" };
    int punteggi[] = { 11, 2, 3, 4, 5, 6, 7, 8, 9, 10, 10, 10, 10 };

    for (int copia = 0; copia < 2; copia++) {
        for (int s = 0; s < 4; s++) {
            for (int v = 0; v < 13; v++) {
                push(top, (char*)semi[s], valori[v], punteggi[v]);
            }
        }
    }
}
```

- Turno del Giocatore (game.c)

```
int turnoGiocatore(Utente* utente, Carta** mazzo, Carta manoGiocatore[], ...) {
    while (exit == 0 && punteggioGiocatore < 21) {
        printMenuGiocatore(* umCarteGiocatore);
        getScelta(&scelta);

        switch (scelta) {
            case 1: // Pesca carta
                cartaEstratta = pop(mazzo);
                manoGiocatore[(*numCarteGiocatore)++] = *cartaEstratta;
                break;
            case 2: // Stai
                exit = 1;
                break;
            case 3: // Raddoppio
                if (*numCarteGiocatore == 2) {
                    utente->saldo -= *puntata;
                    *puntata *= 2;
                    // ... pesca carta e termina turno
                }
                break;
        }
    }
    return punteggioGiocatore;
}
```

Test

Risultati dei Test

Codice Requisito	Codice Test	Nome Test	Descrizione Test	Input	Risultato Atteso	Risultato Ottenuto
R01	1.1	Login Corretto	Credenziali valide	Valide	Accesso consentito	OK
R01	1.2	Login Errato	Credenziali non valide	Invalid	Messaggi o errore	OK
R04	4.1	Nuova Partita	Saldo sufficiente	-	Partita inizia	OK
R07	7.1	Puntata Valida	Importo 50, saldo 1000	50	Puntata accettata	OK
R07	7.2	Puntata Non Valida	Importo 200, saldo 100	200	Messaggi o errore	OK

Conclusioni

- **Punti di Forza**
 - Gestione completa degli utenti con persistenza su database
 - Implementazione corretta delle regole del BlackJack
 - Interfaccia utente intuitiva con messaggi centrati
 - Salvataggio automatico dello stato del gioco
- **Criticità**
 - Mancanza di una GUI grafica
 - Possibili problemi con encoding dei caratteri speciali (semi delle carte)